

Library Management System

Complete Practical Guide — Ubuntu + MySQL + Node.js + Express + Postman

Step 1: Install Required Software

Open Terminal and verify all tools are installed.

Check Node.js

```
node -v
```

Check npm

```
npm -v
```

Check MySQL

```
mysql --version
```

If MySQL is not installed, run:

```
sudo apt update
sudo apt install mysql-server
```

Start MySQL service:

```
sudo systemctl start mysql
sudo systemctl status mysql
```

Step 2: Login to MySQL

```
sudo mysql
```

You should see: mysql>

Step 3: Create Database

```
CREATE DATABASE library_db;
USE library_db;
```

Step 4: Create Tables

Books Table

```
CREATE TABLE Books (
  book_id INT AUTO_INCREMENT PRIMARY KEY,
  title VARCHAR(100),
  author VARCHAR(100),
  category VARCHAR(50),
  available_copies INT
);
```

Students Table

```
CREATE TABLE Students (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  student_name VARCHAR(100),
  department VARCHAR(50),
  email VARCHAR(100)
);
```

Issues Table

```
CREATE TABLE Issues (
  issue_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT,
  book_id INT,
```

```
        issue_date DATE,  
        FOREIGN KEY (student_id) REFERENCES Students(student_id),  
        FOREIGN KEY (book_id) REFERENCES Books(book_id)  
    );
```

Step 5: Verify Tables

```
SHOW TABLES;
```

Expected output: Books Students Issues

Step 6: Insert Books

```
INSERT INTO Books(title, author, category, available_copies)  
VALUES  
( 'Java Programming', 'James Gosling', 'Programming', 5),  
( 'Python Basics', 'Guido Van Rossum', 'Programming', 4),  
( 'DBMS Concepts', 'Korth', 'Database', 3),  
( 'Data Structures', 'Schaum', 'Computer Science', 2),  
( 'Operating System', 'Galvin', 'Computer Science', 6);
```

Verify:

```
SELECT * FROM Books;
```

Step 7: Insert Students

```
INSERT INTO Students(student_name, department, email)  
VALUES  
( 'Rahul', 'CSE', 'rahul@gmail.com'),  
( 'Amit', 'IT', 'amit@gmail.com'),  
( 'Priya', 'CSE', 'priya@gmail.com'),  
( 'Sneha', 'ECE', 'sneha@gmail.com'),  
( 'Rohan', 'MECH', 'rohan@gmail.com');
```

Verify:

```
SELECT * FROM Students;
```

Step 8: Insert Issue Records

```
INSERT INTO Issues(student_id, book_id, issue_date)  
VALUES  
(1,1, '2025-06-01'), (1,2, '2025-06-02'),  
(2,3, '2025-06-03'), (3,1, '2025-06-04'),  
(4,5, '2025-06-05'), (5,4, '2025-06-06'),  
(2,2, '2025-06-07'), (3,3, '2025-06-08'),  
(4,1, '2025-06-09'), (5,5, '2025-06-10');
```

Verify:

```
SELECT * FROM Issues;
```

Step 9: Exit MySQL

```
exit
```

Step 10: Create Node Project

```
mkdir library-api  
cd library-api
```

Step 11: Initialize & Install Packages

```
npm init -y  
npm install express mysql2 cors
```

Step 12: Create server.js

```
touch server.js
```

Step 13: Write Complete server.js Code

Open the file with: nano server.js — then paste the full code below, save with CTRL+X → Y → ENTER

Base Server Setup (Paste first)

```
const express = require('express');
const mysql    = require('mysql2');
const cors     = require('cors');

const app = express();
app.use(express.json());
app.use(cors());

const db = mysql.createConnection({
  host:     'localhost',
  user:     'root',
  password: '',
  database: 'library_db'
});

db.connect(err => {
  if (err) { console.log(err); }
  else     { console.log('MySQL Connected'); }
});
```

API 1 — POST /books (Add a Book)

```
app.post('/books', (req, res) => {
  const { title, author, category, available_copies } = req.body;
  db.query(
    'INSERT INTO Books(title,author,category,available_copies) VALUES (?, ?, ?, ?)',
    [title, author, category, available_copies],
    (err, result) => {
      if (err) return res.status(500).json(err);
      res.json({ message: 'Book Added Successfully' });
    }
  );
});
```

API 2 — GET /books (Get All Books)

```
app.get('/books', (req, res) => {
  db.query('SELECT * FROM Books', (err, result) => {
    if (err) return res.status(500).json(err);
    res.json(result);
  });
});
```

API 3 — POST /issues (Issue a Book)

```
app.post('/issues', (req, res) => {
  const { student_id, book_id } = req.body;
  db.query(
    'INSERT INTO Issues(student_id,book_id,issue_date) VALUES (?, ?, CURDATE())',
    [student_id, book_id],
    (err, result) => {
      if (err) return res.status(500).json(err);
      res.json({ message: 'Book Issued Successfully' });
    }
  );
});
```

API 4 — GET /students/:id/issues (Student Issue History)

```
app.get('/students/:id/issues', (req, res) => {
  const id = req.params.id;
  db.query(
    `SELECT
      Students.student_name,
      Books.title,
```

```
      Issues.issue_date
FROM Issues
JOIN Students ON Issues.student_id = Students.student_id
JOIN Books    ON Issues.book_id    = Books.book_id
WHERE Students.student_id = ?`,
[id],
(err, result) => {
  if (err) return res.status(500).json(err);
  res.json(result);
}
);
});
```

Start the Server (at the end of file)

```
app.listen(3000, () => {
  console.log('Server Running on Port 3000');
});
```

Step 14: Run the Server

node server.js

Expected: MySQL Connected Server Running on Port 3000

Step 15: Test All APIs with Postman

API 1 — Add Book [POST]

URL: `http://localhost:3000/books`

Body: raw → JSON

```
{
  "title": "C Programming",
  "author": "Dennis",
  "category": "Programming",
  "available_copies": 5
}
```

Expected response: { "message": "Book Added Successfully" }

API 2 — Get All Books [GET]

URL: `http://localhost:3000/books`

Expected response: JSON array of all books

API 3 — Issue a Book [POST]

URL: `http://localhost:3000/issues`

Body: raw → JSON

```
{
  "student_id": 1,
  "book_id": 2
}
```

Expected response: { "message": "Book Issued Successfully" }

API 4 — Student Issue History [GET]

URL: `http://localhost:3000/students/1/issues`

Expected response: [{ "student_name": "Rahul", "title": "Java Programming", "issue_date": "2025-06-01" }]

Exam Checklist — What to Show the Examiner

1. Database created — run: `SHOW TABLES;`
2. All 3 tables visible: Books, Students, Issues
3. Sample data inserted — run: `SELECT * FROM Books;`
4. Run: `SELECT * FROM Students;`
5. Run: `SELECT * FROM Issues;`
6. Start Node server: `node server.js`
7. Postman — demonstrate all 4 APIs with request & response

This guide covers everything needed to score full marks in the Library Management System practical.